

Chapitre 1 : Mesure des performances

Objectif général

La mesure des performances permet d'évaluer, comparer et améliorer l'efficacité d'un ordinateur. Le critère le plus important est le **temps d'exécution** d'un programme.

1. Pourquoi mesurer les performances ?

- Détecter les **faiblesses** d'un système.
- **Comparer** différents processeurs.
- **Optimiser** les performances matérielles et logicielles.

2. Évolution des ordinateurs

- **Avant 1945** : machines mécaniques, lentes et peu fiables.
- **Après 1945** : électronique, signaux électriques → exécution beaucoup plus rapide.

3. Facteurs influençant la performance

a) Côté logiciel

- **Jeu d'instructions** :
 - **CISC** : instructions complexes, plusieurs cycles.
 - **RISC** : instructions simples, peu de cycles.
- **Compilateur** : influence le nombre d'instructions exécutées et l'efficacité globale.

b) Côté matériel

- **Architecture** du processeur.
- **Fréquence d'horloge**.
- **Technologie** utilisée (circuits, mémoire, cache).

4. Évaluation des performances

- **Côté utilisateur** : temps nécessaire pour exécuter une tâche (temps de réponse).
- **Côté système** : nombre de tâches exécutées par unité de temps (débit).
- Améliorer le temps de réponse améliore souvent le débit, mais pas toujours l'inverse.

5. Temps CPU (critère principal)

Le temps CPU dépend de :

- La **durée d'un cycle d'horloge**.
- Le **nombre d'instructions exécutées**.
- Le **nombre moyen de cycles par instruction**.

6. Performance en MIPS

- Mesure le nombre de millions d'instructions exécutées par seconde.
- Indicateur **peu fiable seul**, car il dépend du jeu d'instructions.

7. Rendement (Throughput)

- Nombre de programmes exécutés par unité de temps.
- Unité : **programmes/seconde**.

8. Remarques importantes

- Le nombre d'instructions dépend du **compilateur** et du **jeu d'instructions**.
- Le nombre de cycles par instruction dépend de l'**architecture**.
- La durée du cycle dépend du **matériel**.
- **Le meilleur indicateur global reste toujours le temps d'exécution**

Résumé des principales formules

Nom	Formule	Équation
Fréquence d'horloge	$f = \frac{1}{\tau}$	
Temps CPU principal	$T = Ic \times CPI \times \tau = \frac{Ic \times CPI}{f}$	(Eq.2)
Temps avec mémoire	$T = Ic \times (p + m \times k) \times \tau$	(Eq.3)
Temps CPU (cycles totaux)	$T = C \times \tau = \frac{C}{f}$	(Eq.4)
Moyenne des cycles	$CPI = \frac{C}{Ic}$	(Eq.5)
Vitesse MIPS	$MIPS = \frac{f}{CPI \times 10^6} = Ic / (T \times 10^6) = (f \times Ic) / (c \times 10^6)$	(Eq.7)
Temps via MIPS	$T = Ic \times \frac{10^{-6}}{MIPS}$	(Eq.8)
Rendement	$Wp = \frac{f}{Ic \times CPI} = (MIPS) \times ((10^6) / (Ic))$	(Eq.9)

Chapitre 2 : Programmation en langage de bas niveau (Assembleur)

1. Objectif du chapitre

Ce chapitre explique comment exploiter efficacement un processeur grâce à la **programmation en assembleur**, un langage proche du matériel.

Il permet de comprendre :

- L'exécution des instructions.
- L'utilisation des registres.
- La gestion de la mémoire.
- L'impact du code sur les performances.

2. Langage machine et assembleur

- **Langage machine** : code binaire propre à chaque processeur.
- **Langage assembleur** : version lisible du langage machine utilisant des mnémoniques (ADD, SUB, LW...).
- Programmer en assembleur nécessite de connaître l'**architecture du processeur**.

3. Architecture externe d'un processeur

Un programmeur doit connaître :

- **Registres visibles** : nombre et taille.
- **Adressage mémoire** : bus de données, bus d'adresses, modes d'adressage.
- **Jeu d'instructions** : formats et types d'instructions.
- **Interruptions et exceptions** : mécanismes et registres associés.

4. Intérêt des langages de bas niveau

- Contrôle total du matériel.
- Exécution rapide.
- Faible consommation mémoire.
- Meilleure compréhension interne (optimisation, débogage).
- Adapté aux systèmes embarqués et critiques.

5. Étude de cas : processeur MIPS R3000

- Processeur **32 bits**, architecture **RISC**.
- Deux modes :
 - **Utilisateur**
 - **Superviseur**
- Utilisé dans stations de travail et systèmes embarqués.

6. Registres du MIPS R3000

a) Registres non protégés

- **32 registres généraux** (\$R0 à \$R31)
 - \$R0 = 0
 - \$R31 : adresse de retour
- **PC** : compteur de programme
- **HI / LO** : résultats de multiplication/division

b) Registres protégés (coprocesseur système)

- **SR** : état et interruptions
- **CR** : cause des exceptions
- **EPC** : adresse de retour après exception
- **BAR** : adresse fautive

7. Organisation de la mémoire

- Bus de données et d'adresses : **32 bits**
- Mémoire organisée par **octets**
- Instructions sur **32 bits**
- Données : byte (8 bits), half-word (16 bits), word (32 bits)
- Adresses **alignées**
- Architecture **Little Endian**
- Mémoire divisée en :
 - Segment utilisateur
 - Segment système

8. Structure d'un programme MIPS

- Section instructions
- Section données
- Pile (appels de fonctions)

9. Modes d'adressage MIPS

Utilisés par le MIPS R3000 :

- Immédiat
- Par registre
- Basé (registre + déplacement)
- Relatif (branchements)

10. Formats d'instructions

Toutes les instructions font **32 bits** :

- **Format R** : opérations registre–registre
- **Format I** : immédiat, accès mémoire, branchements
- **Format J** : sauts longs

MIPS est une architecture **Load/Store** :

- Seules les instructions **Load** et **Store** accèdent à la mémoire.
- Les autres utilisent uniquement les registres.

11. Instructions principales

Accès mémoire

LB, LBU, LH, LHU, LW

SB, SH, SW

Arithmétiques et logiques

ADD / ADDU, SUB / SUBU

MULT / MULTU, DIV / DIVU

AND, OR

Instructions immédiates

LUI, ORI

Branchement et saut

BEQ, BGTZ

J, JAL, JR

12. Données manipulées

- **Entiers** : complément à 2
- **Caractères** : ASCII (8 bits)
- **Flottants** : coprocesseur 1 (IEEE-754)
- **Chaînes de caractères** : généralement format C (terminées par 0)

Chapitre 3 : Écriture de programmes en langage d'assemblage MIPS

1. Assembleur MIPS et conventions

- L'assembleur MIPS permet d'écrire des programmes pour le **processeur MIPS-R3000**.
- Des **conventions d'utilisation des registres** existent pour assurer la compatibilité entre programmes.
- Elles facilitent les appels de fonctions et la collaboration entre programmeurs.

Conventions principales des registres

- **\$a0 – \$a3** : passage des 4 premiers paramètres.
- **\$v0, \$v1** : valeurs de retour.
- **\$t0 – \$t9** : registres temporaires (non sauvegardés).
- **\$s0 – \$s7** : registres sauvegardés par les fonctions.
- **\$sp** : pointeur de pile.
- **\$fp** : pointeur de cadre (variables locales).
- **\$ra** : adresse de retour.
- **\$at, \$k0, \$k1** : réservés (ne pas utiliser).

2. Directives d'assemblage et pseudo-instructions

Directives

- **.data** : section données.
- **.text** : section programme.
- **.byte, .half, .word** : données entières.
- **.ascii, .asciiz** : chaînes de caractères.
- **.float, .double** : données flottantes.

Pseudo-instructions

- **LA** : charger une adresse.
- **LI** : charger une constante.
- **MOVE** : copier un registre.
- **MUL** : multiplication simplifiée.

Elles sont traduites par l'assembleur en instructions réelles.

3. Simulateur SPIM

- SPIM simule l'exécution des programmes MIPS.
- Structure minimale d'un programme :
 - Section données
 - Section programme
 - Étiquette **main**
 - Fin avec **jr \$ra** ou **syscall 10**

4. Exemples de programmes

a) Addition de deux entiers

- Chargement des données depuis la mémoire.
- Opérations arithmétiques uniquement sur registres.
- Sauvegarde du résultat en mémoire.
- Utilisation de l'adressage basé (déplacement).

b) Somme des éléments d'un tableau

- Utilisation d'une boucle.
- Registres pour :
 - Adresse du tableau
 - Compteur
 - Somme partielle
- Accès séquentiel par incrément de l'adresse (+4).

5. Interruptions et exceptions

Types d'événements

- **Exceptions** : erreurs internes (overflow, adresse illégale...).
- **Interruptions** : événements externes.
- **SYSCALL** : appels système.
- **RESET** : réinitialisation.

Traitement général

- Passage en mode superviseur.
- Sauvegarde du contexte (PC, SR).
- Indication de la cause.
- Saut vers le gestionnaire (0x80000180).

6. Appels système (SYSCALL)

Permettent les entrées/sorties :

- Afficher un entier (1)
- Lire un entier (5)
- Afficher une chaîne (4)
- Lire une chaîne (8)
- Quitter le programme (10)

Les arguments sont passés par **\$a0**, **\$a1** et le numéro par **\$v0**.

7. Signal RESET

- Interruption **non masquable**.
- Force l'exécution d'un programme de réinitialisation.
- Adresse de départ : **0xBFC00000**.
- Pas de sauvegarde d'adresse de retour.
- Passage en mode superviseur et masquage des interruptions.

Chapitre 4 : Procédures et fonctions en MIPS

1. Rôle des procédures

- Les procédures servent à **structurer les programmes**.
- Elles s'exécutent sur le **même processeur** que le programme appelant et **partagent les registres**.
- Une procédure :
 - reçoit des paramètres,
 - effectue un traitement,
 - retourne un résultat,
 - rend le contrôle au programme appelant.
- Une même procédure peut être appelée plusieurs fois depuis différents endroits.

2. La pile en MIPS

- La pile est une zone mémoire **LIFO** (Last In, First Out).
- Le registre **\$sp (\$29)** pointe vers le sommet de la pile.
- La pile sert à :
 - sauvegarder l'adresse de retour,
 - sauvegarder les registres,
 - passer des paramètres supplémentaires,
 - stocker les variables locales.
- MIPS utilise un registre général (**\$sp**) comme pointeur de pile.

3. Appel et retour de procédure

- **Appel** : **jal procedure**
 - saute vers la procédure,
 - sauvegarde l'adresse de retour dans **\$ra (\$31)**.
- **Retour** : **jr \$ra**
 - retourne à l'instruction suivant l'appel.

4. Passage des paramètres et valeurs de retour

- Paramètres :
 - 4 premiers → **\$a0 à \$a3**
 - supplémentaires → **pile**
- Valeurs de retour :

- 1ère et 2ème → **\$v0, \$v1**
- supplémentaires → **pile**

5. Intégrité des registres

- Appelant et procédure utilisent les **mêmes registres**, ce qui peut provoquer des erreurs.
- Règle fondamentale :
 - Les registres **\$S0 à \$S7** doivent être **sauvegardés et restaurés** par la procédure si elle les utilise.
 - Les registres **\$t0 à \$t9** sont temporaires et **non sauvegardés**.
- La sauvegarde/restauration se fait **via la pile**.

6. Exemple simple (une fonction)

- Les paramètres sont passés par **\$a0–\$a3**.
- Le résultat est retourné dans **\$v0**.
- Les registres **\$S** utilisés par la fonction sont sauvegardés dans la pile puis restaurés avant **jr \$ra**.

7. Appels imbriqués

- Une fonction peut appeler une autre fonction.
- Dans ce cas, il faut sauvegarder :
 - les registres **\$S** utilisés,
 - les paramètres si nécessaire,
 - **\$ra** (adresse de retour).
- La pile permet de préserver le **contexte** de chaque appel.

8. Idées clés à retenir

- **jal** → appel de procédure
- **jr \$ra** → retour
- **\$a0–\$a3** → paramètres
- **\$v0** → valeur de retour
- **\$S0–\$S7** → registres à préserver
- **\$t0–\$t9** → registres temporaires
- **La pile est indispensable** pour garantir le bon fonctionnement des procédures, surtout avec des appels imbriqués.